

Lab 06: Real-Time Clock using SysTick (in C)

ECE0202: Embedded Processors and Interfacing

Spring 2026

Stefan Zhang.....Student ID 2024141520349

Knight LiuStudent ID 2024141520361

Date Submitted: June 15, 2026

1) Pre-Lab

1.1) The registers related to the RTC module

The registers related to the RTC module are shown below. The registers that we care about in this lab are RTC_TR, RTC_ISR, and RTC_WPR. The other registers, however, may be used for other related tasks.

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x00	RTC_TR	Res	Res	Res	Res	Res	Res	Res	Res	Res	PM	HT [1:0]	HU[3:0]			Res	MNT[2:0]			MNU[3:0]			Res	ST[2:0]		SU[3:0]									
	Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
0x04	RTC_DR	Res	Res	Res	Res	Res	Res	Res	Res	Res	YT[3:0]			YU[3:0]			WDU[2:0]		MT	MU[3:0]			Res	Res	DT [1:0]		DU[3:0]								
	Value																																		
0x08	RTC_CR	Res	Res	Res	Res	Res	Res	Res	Res	ITSE	COE	OSL [1:0]	POL	COSEL	BKP	SUB1H	ADD1H	TSIE	WUTIE	ALRBIE	ALRAIE	TSE	WUTE	ALRBE	ALRAE	Res	FMT	BYPHAD	REFCKON	TSEDGE	WUCKSEL[2:0]				
	Value																																		
0x0C	RTC_ISR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	RECALPF	TAMP3F	TAMP2F	TAMP1F	TSOVF	TSF	WUTF	ALRBF	ALRAF	INIT	INITF	RSF	INITS	SHPF	WUTFWF	ALRBUF	ALRAWF	
	Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x10	RTC_PRER	Res	Res	Res	Res	Res	Res	Res	Res	Res	PREDIV_A[6:0]						PREDIV_S[14:0]																		
	Value																																		
0x14	RTC_WUTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	WUT[15:0]																	
	Value																																		
0x1C	RTC_ALRMAR	MSK4	WDSEL	DT [1:0]		DU[3:0]			MSK3		PM	HT [1:0]	HU[3:0]			MSK2		MNT[2:0]		MNU[3:0]			MSK1		ST[2:0]		SU[3:0]								
	Value																																		
0x20	RTC_ALRMBR	MSK4	WDSEL	DT [1:0]		DU[3:0]			MSK3		PM	HT [1:0]	HU[3:0]			MSK2		MNT[2:0]		MNU[3:0]			MSK2		ST[2:0]		SU[3:0]								
	Value																																		

[illegible][illegible]

1.2) SysTick Reload Value

The SysTick clock frequency is 80 MHz after calling `System_Clock_Init()`. The required interrupt period is 1 ms.

$$Interrupt\ Period = (1 + Value) \times \frac{1}{Timer\ Frequency}$$

$$1 \times 10^{-3} = (1 + Value) \times \frac{1}{80 \times 10^6}$$

$$Value = 1 \times 10^{-3} \times 80 \times 10^6 - 1 = 79999.$$

Therefore,

$$SysTick \rightarrow LOAD = 79999.$$

2) Experimental Setup

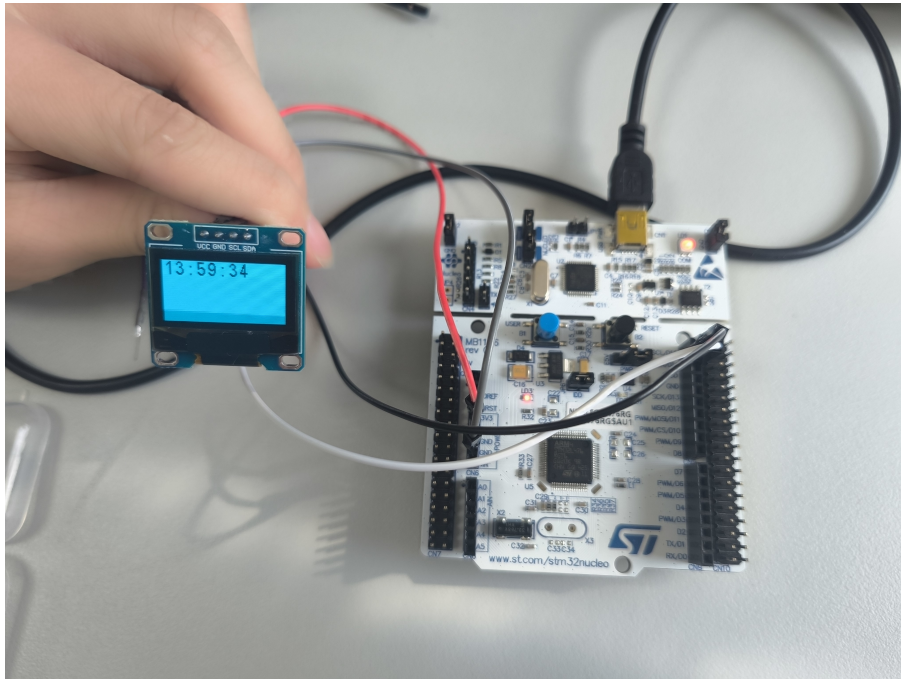


Figure 1: Hardware connection of the stepper motor circuit

3) Code

The main C program used in this experiment is shown below.

```
#include "stm321476xx.h"
#include "SysClock.h"
#include "UART.h"
```



```

#include "I2C.h"
#include "RTC.h"
#include "ssd1306.h"
#include <string.h>
#include <stdio.h>

void System_Clock_Init(void);
void RTC_Clock_Init(void);
void I2C_GPIO_init(void);
void DisplayString(char* message);
static uint32_t RTC_BcdToDec(uint32_t value);
volatile uint32_t msTicks;
char time[] = "13:59:30";

void DisplayString(char* message){

    ssd1306_Fill(White);
    ssd1306_SetCursor(2,0);
    ssd1306_WriteString(message, Font_11x18, Black);
    ssd1306_UpdateScreen();

}

static uint32_t RTC_BcdToDec(uint32_t value){
    return ((value >> 4) * 10U) + (value & 0x0FU);
}

void SysTick_Handler(void){
    msTicks++;

    if (msTicks >= 1000U) {
        msTicks = 0U;
        sprintf(time, "%02lu:%02lu:%02lu",
            RTC_BcdToDec(RTC_TIME_GetHour()),
            RTC_BcdToDec(RTC_TIME_GetMinute()),
            RTC_BcdToDec(RTC_TIME_GetSecond()));
        DisplayString(time);
    }
}

void delay (uint32_t T){
    uint32_t curTicks;

    curTicks = msTicks;
    while ((msTicks - curTicks) < T) {
    }
}

int main(void){
    System_Clock_Init();
    I2C_GPIO_init();
    I2C_Initialization(I2C1);
    ssd1306_Init();
    RTC_Clock_Init();

    RTC_Disable_Write_Protection();
    RTC->ISR |= RTC_ISR_INIT;
    while ((RTC->ISR & RTC_ISR_INITF) == 0U) {
    }
}

```

```

    RTC_Set_Time(0U, 0x13, 0x59, 0x30);

    RTC->ISR &= ~RTC_ISR_INIT;
    while ((RTC->ISR & RTC_ISR_RSF) == 0U) {
    }
    RTC_Enable_Write_Protection();

    msTicks = 0U;
    SysTick->CTRL = 0U;
    SysTick->LOAD = 80000U - 1U;
    SysTick->VAL = 0U;
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk |
                    SysTick_CTRL_ENABLE_Msk;

    NVIC_SetPriority(SysTick_IRQn, 1);
    NVIC_EnableIRQ(SysTick_IRQn);

    DisplayString(time);

    // Dead loop & program hangs here
    while(1){
    }
}

```

4) Participation and Contribution

Table 1: Participation and contribution of group members

Name	Participation and Contribution
Stefan Zhang	Code programming; testing; report writing
Knight Liu	Hardware connection; circuit testing